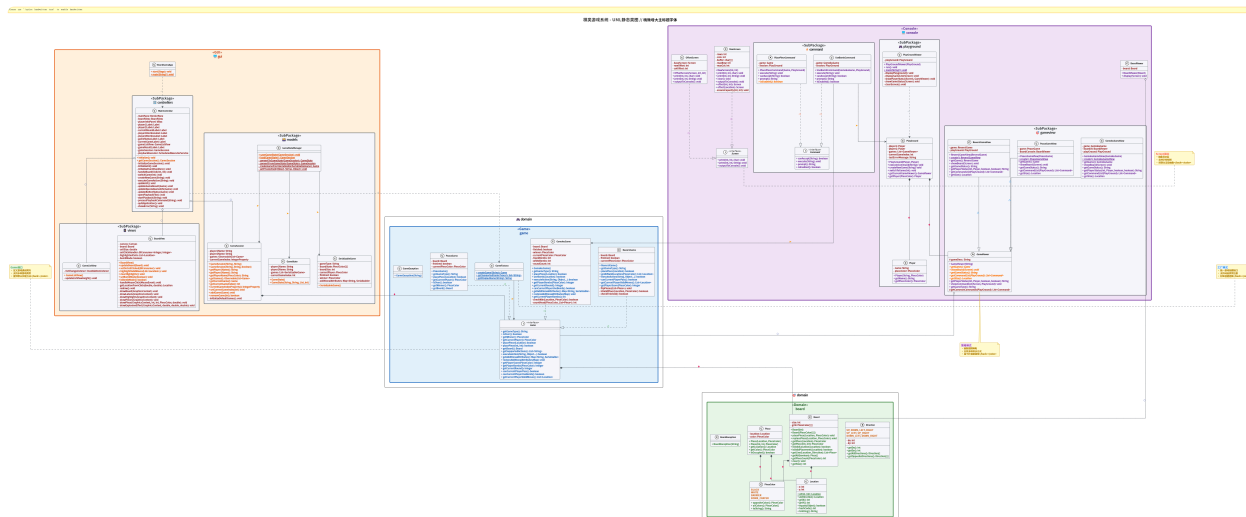


Readme

运行方法

在pj 目录下运行`java -jar target\board-games-1.0-SNAPSHOT.jar`

使用图形给出代码结构说明



(原图可见该文件同目录下的附件attachment)

项目源代码文件概览及主要功能

控制台应用模块 (`console/`)

核心显示组件

- `BoardViewer.java` : 控制台棋盘显示工具类, 支持多种棋子类型 (黑白棋、障碍物、弹坑) 的字符渲染

命令系统 (`command/`)

- `Command.java` : 命令模式接口, 定义命令的执行、启用状态检查和提示信息
- `NewGameCommand.java` : 新游戏创建命令, 支持和平棋、黑白棋、五子棋类型选择
- `PassCommand.java` : 跳过回合命令, 主要用于黑白棋游戏中无法落子的情况
- `PlacePieceCommand.java` : 落子命令, 解析坐标输入并在棋盘上放置棋子
- `PlaybackCommand.java` : 回放命令实现, 从文件读取并自动执行游戏命令序列
- `QuitCommand.java` : 退出程序命令实现
- `SelectGameCommand.java` : 游戏切换命令, 允许在多个游戏实例间切换

- `UseBombCommand.java` : 炸弹使用命令, 专用于五子棋游戏的特殊功能

游戏视图层 (`gameview/`)

- `GameViewer.java` : 游戏视图抽象基类, 定义显示框架和通用命令管理
- `GomokuGameView.java` : 五子棋控制台视图, 包含炸弹功能和回合数显示
- `PeaceGameView.java` : 和平棋控制台视图实现, 支持基本落子操作
- `ReversiGameView.java` : 黑白棋控制台视图, 包含合法落子位置提示和分数显示

游戏环境管理 (`playground/`)

- `PlayGround.java` : 控制台游戏环境核心逻辑, 管理多游戏切换、命令执行和回放模式
- `PlayGroundViewer.java` : 控制台主界面显示器, 整合棋盘、玩家信息、游戏列表和命令提示
- `Player.java` : 玩家数据类, 包含姓名和棋子颜色信息

屏幕输出系统 (`screen/`)

- `RawScreen.java` : 控制台屏幕缓冲区实现, 支持动态扩容和偏移显示
- `OffsetScreen.java` : 屏幕偏移代理类, 用于相对位置显示
- `Screen.java` : 屏幕操作接口, 定义字符和字符串输出方法

领域模型层 (`domain/`)

棋盘模型 (`board/`)

- `Board.java` : 棋盘核心领域对象, 提供棋子放置、位置验证、连线获取和统计功能
- `BoardException.java` : 棋盘操作异常类, 处理无效位置等错误情况
- `Direction.java` : 方向枚举类, 定义八个方向的偏移量
- `Location.java` : 位置坐标类, 表示棋盘上的行列位置
- `Piece.java` : 棋子数据类, 包含位置和颜色信息
- `PieceColor.java` : 棋子颜色枚举, 定义黑白棋子、障碍物和弹坑类型

游戏逻辑 (`game/`)

- `Game.java` : 游戏逻辑抽象基类, 定义通用的游戏操作接口
- `GameException.java` : 游戏逻辑异常类, 处理游戏规则违反等错误
- `GameFactory.java` : 游戏工厂类, 根据类型字符串创建对应的游戏实例
- `GomokuGame.java` : 五子棋游戏逻辑实现, 包含连珠检测和炸弹功能
- `PeaceGame.java` : 和平棋游戏逻辑实现, 棋盘填满即结束的简单游戏

- `ReversiGame.java` : 黑白棋游戏逻辑实现，包含翻转规则和合法位置计算

图形用户界面模块 (`gui/`)

应用入口

- `BoardGameApp.java` : JavaFX 主应用类，负责 FXML 加载、CSS 样式应用和窗口初始化
- `BoardGameAppLauncher.java` : JavaFX 应用启动器，解决模块化启动问题

控制器层 (`controllers/`)

- `MainController.java` : JavaFX 主控制器，处理用户交互、界面更新、游戏状态管理和回放功能

数据模型层 (`models/`)

- `GameSession.java` : GUI 游戏会话管理类，维护玩家信息和游戏列表
- `GameState.java` : 游戏会话状态序列化数据类，用于保存和恢复完整游戏状态
- `GameStateManager.java` : 游戏状态持久化管理器，实现游戏状态的文件保存和加载
- `SerializableGame.java` : 单个游戏序列化数据类，包含棋盘状态和游戏特定属性

视图组件层 (`views/`)

- `BoardView.java` : JavaFX 棋盘自定义组件，支持多种棋子渲染、高亮提示和炸弹模式交互
- `GameListView.java` : JavaFX 游戏列表自定义组件，动态调整高度并显示游戏类型

资源文件目录 (`resources/`)

样式资源 (`css/`)

- `styles.css` : JavaFX 界面样式定义，包含按钮、列表和玩家高亮等样式规则

界面布局 (`fxml/`)

- `main.fxml` : JavaFX 主界面布局文件，定义左侧棋盘区域和右侧三列功能区域的完整界面结构

关键代码及其设计思路

1. 游戏核心接口 (Game.java)

```
// ...existing code...
public interface Game {
    String getGameType();
    boolean isOver();
    PieceColor getWinner();
    PieceColor getCurrentPlayer();
    boolean placePiece(Location loc);
    Board getBoard();
    List<Location> getCurrentPlayerValidMoves(); // 获取当前玩家可落子位置
    // ... 获取游戏特有属性、执行特殊操作等 ...
    default Map<String, Serializable> getAdditionalAttributes() { return
Collections.emptyMap(); }
    default void restoreAdditionalAttributes(Map<String, Serializable>
attributes) {}
    default boolean canCurrentPlayerPass() { return false; }
    default boolean canCurrentPlayerUseBomb() { return false; }
    default Integer getPlayerBombs(PieceColor playerColor) { return null; }
    default Integer getCurrentRound() { return null; }
}
```

设计思路: 定义统一的游戏契约，使得上层代码（如UI、会话管理）能以一致的方式与不同类型的游戏交互。通过默认方法提供可选功能的扩展点，如获取可落子位置、特殊属性、是否可跳过/使用炸弹等，便于具体游戏类按需实现。

2. 游戏工厂 (GameFactory.java)

```
// ...existing code...
public class GameFactory {
    private static final Map<String, GameInfo> gameRegistry = new HashMap<>
();

    static {
        // 注册所有支持的游戏及其构造函数引用
        register("peace", "和平棋", PeaceGame::new);
        register("reversi", "黑白棋", ReversiGame::new);
        register("gomoku", "五子棋", GomokuGame::new);
    }

    public static void register(String type, String description,
```

```

Supplier<Game> creator) {
    gameRegistry.put(type.toLowerCase(), new GameInfo(type, description,
creator));
}

public static Game createGame(String gameType) {
    GameInfo gameInfo = gameRegistry.get(gameType.toLowerCase());
    if (gameInfo != null) {
        return gameInfo.getCreator().get();
    }
    throw new IllegalArgumentException("不支持的游戏类型: " + gameType);
}
// ...existing code...
}

```

设计思路: 应用工厂模式，将游戏对象的创建过程封装起来。通过注册机制，可以方便地添加新的游戏类型，而无需修改工厂本身的核心创建逻辑，实现了创建与使用的解耦。

3. GUI控制器交互 (MainController.java)

```

// ...existing code...
public class MainController {
    // ... @FXML 注入的UI元素 ...
    private GameSession gameSession;

    private void handleBoardClick(int row, int col) {
        try {
            Game currentGame = gameSession.getCurrentGame();
            if (currentGame == null || currentGame.isOver()) return;

            if (boardView.isBombMode() &&
currentGame.getGameType().equals("gomoku")) {
                if (currentGame instanceof domain.game.GomokuGame gomokuGame)
{
                    gomokuGame.useBomb(new Location(row, col));
                }
                boardView.setBombMode(false); // 退出炸弹模式
            } else {
                currentGame.placePiece(new Location(row, col));
            }
            updateUI(); // 更新界面显示
        } catch (GameException e) {

```

```

        showError(e.getMessage());
    }
}

private void updateUI() {
    Game currentGame = gameSession.getCurrentGame();
    if (currentGame == null) return;
    // ... 更新棋盘视图 ...
    boardView.updateBoard(currentGame.getBoard());
    // ... 更新玩家信息、游戏状态、按钮状态等 ...
    updatePlayerInfo(currentGame);
    updateGameResult(currentGame);
    updateButtonStatus(currentGame);
    updateSpecialGameInfo(currentGame);
}
// ...existing code...
}

```

设计思路: `MainController` 作为GUI的协调者。`handleBoardClick` 方法响应棋盘点击事件，根据当前是否为炸弹模式，调用游戏对象的 `placePiece` 或 `useBomb` 方法。之后调用 `updateUI` 方法，该方法从 `GameSession` 和当前 `Game` 对象获取最新状态，并更新棋盘视图 (`boardView`)、玩家信息标签、游戏结果等所有相关的UI元素。

4. 自定义棋盘绘制 (`BoardView.java`)

```

// ...existing code...
public class BoardView extends Pane {
    private Canvas canvas;
    private Board board;
    // ... cellSize, cellClickHandler, highlightedCells, bombMode ...

    private void redraw() {
        if (board == null) return;
        GraphicsContext gc = canvas.getGraphicsContext2D();
        double width = getWidth();
        double height = getHeight();
        gc.clearRect(0, 0, width, height); // 清除画布

        this.cellSize = Math.min(width, height) / (board.getSize() + 2.0); //
动态计算单元格大小
        canvas.setWidth(cellSize * (board.getSize() + 2));
        canvas.setHeight(cellSize * (board.getSize() + 2));
    }
}

```

```

        drawBoard(gc);    // 绘制棋盘背景和网格
        drawLabels(gc);  // 绘制行列标签
        drawPieces(gc);   // 绘制棋子
        drawHighlights(gc); // 绘制高亮提示
    }

    private void drawPiece(GraphicsContext gc, int row, int col, PieceColor
pieceColor, double size) {
        double x = cellSize * (col + 1) + (cellSize - size) / 2;
        double y = cellSize * (row + 1) + (cellSize - size) / 2;

        switch (pieceColor) {
            case BLACK:
                gc.setFill(Color.BLACK);
                gc.fillOval(x, y, size, size);
                break;
            case WHITE:
                gc.setFill(Color.IVORY); // 使用象牙白以便区分
                gc.fillOval(x, y, size, size);
                gc.setStroke(Color.BLACK); // 给白棋加边框
                gc.strokeOval(x, y, size, size);
                break;
            case BARRIER:
                gc.setFill(Color.DARKGRAY);
                gc.fillRect(x, y, size, size);
                break;
            case BOMB_CRATER: // 弹坑效果
                gc.setFill(Color.LIGHTGRAY); // 弹坑底色
                gc.fillOval(x, y, size, size);
                drawExplosionEffect(gc, x + size/2, y + size/2, size/2 *
0.7);
                break;
        }
    }

    private void drawExplosionEffect(GraphicsContext gc, double centerX,
double centerY, double rayLength) {
        gc.setStroke(Color.ORANGERED);
        gc.setLineWidth(2);
        for (int i = 0; i < 8; i++) { // 绘制8条辐射线
            double angle = i * Math.PI / 4;
            gc.strokeLine(centerX, centerY, centerX + rayLength *

```

```

Math.cos(angle), centerY + rayLength * Math.sin(angle));
    }
}
// ...existing code...
}

```

设计思路: `BoardView` 是一个自定义的JavaFX `Pane`，它封装了棋盘的渲染逻辑。`redraw` 方法是核心绘制入口，它会调用一系列私有方法分别绘制棋盘网格、行列标签、棋子和高亮提示。`drawPiece` 根据棋子类型 (`PieceColor`) 绘制不同颜色和形状的棋子，其中 `BOMB_CRATER` 会调用 `drawExplosionEffect` 来绘制爆炸后的效果，增强了视觉表现力。

5. 游戏状态持久化(`GameStateManager.java`)

```

// ...existing code...
public class GameStateManager {
    private static final String SAVE_FILE = "pj.game"; // 保存文件名

    public static void saveGameState(GameSession gameSession) {
        if (gameSession == null) return;
        try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream(SAVE_FILE))) {
            GameState gameState = convertToGameState(gameSession); // 将
GameSession转换为可序列化的GameState
            oos.writeObject(gameState);
        } catch (IOException e) {
            System.err.println("保存游戏状态失败: " + e.getMessage());
        }
    }

    public static GameSession loadGameState() {
        File file = new File(SAVE_FILE);
        if (!file.exists()) return null; // 文件不存在则不加载

        try (ObjectInputStream ois = new ObjectInputStream(new
FileInputStream(SAVE_FILE))) {
            GameState gameState = (GameState) ois.readObject();
            return convertFromGameState(gameState); // 将GameState转换回
GameSession
        } catch (IOException | ClassNotFoundException e) {
            System.err.println("加载游戏状态失败: " + e.getMessage());
            file.delete(); // 如果加载失败，可能文件已损坏，删除之

```

```

        return null;
    }
}
// ... convertToGameState 和 convertFromGameState 方法辅助转换 ...
}

```

设计思路: 使用Java的序列化机制来实现游戏状态的保存和加载。`saveGameState` 方法将当前的 `GameSession` 对象（通过一个可序列化的 `GameState` DTO 中转，`GameState` 包含 List 和其他会话信息）写入到指定文件。`loadGameState`方法从该文件读取并反序列化对象，恢复之前的游戏会话。`SerializableGame`类用于存储单个游戏的核心状态（如棋盘布局、当前玩家、附加属性等）。

6. 控制台命令接口 (`Command.java`)

```

package console.command;

public interface Command {
    boolean canAccept(String input); // 判断输入是否匹配此命令
    void execute(String input);      // 执行命令
    String prompt();                 // 获取命令的提示信息
    boolean isEnabled();             // 命令当前是否可用
}

```

设计思路: 应用命令模式。`Command` 接口定义了所有控制台命令的统一行为：`canAccept` 用于解析用户输入，`execute` 用于执行命令逻辑，`prompt` 用于向用户显示命令格式，`isEnabled` 判断命令在当前上下文是否可用。这种设计使得添加新命令或修改现有命令更加方便和模块化。

7. 控制台UI渲染 (`PlayGroundViewer.java`)

```

// filepath:
d:\project\vscode\java_lab\pj\lab\console\playground\PlayGroundViewer.java
// ...existing code...
public class PlayGroundViewer {
    private final PlayGround playGround;

    // ...构造函数...

    private void displayPlayground() {
        GameViewer gameViewer = playGround.getCurrentGameViewer();
    }
}

```

```

    if (gameViewer == null) {
        System.out.println("没有活动的游戏。");
        // ... 显示创建游戏的提示 ...
        return;
    }
    // 使用一个虚拟屏幕对象来组织输出内容
    RawScreen screen = new RawScreen(50, 120); // 假设的屏幕大小

    // 1. 显示棋盘状态
    gameViewer.showBoard(screen.offset(0, 0));

    // 2. 显示玩家状态（玩家列表、当前玩家、分数、炸弹数等）
    // 假设 gameViewer.getSize() 返回棋盘在屏幕上的尺寸
    int boardDisplayHeight = gameViewer.getSize().getY();
    int boardDisplayWidth = gameViewer.getSize().getX() * 2 + 4; // 估算棋
    盘宽度
    showPlayerStatus(screen.offset(1, boardDisplayWidth + 2), gameViewer);

    // 3. 显示游戏列表
    displayGameList(screen.offset(boardDisplayHeight / 2 + 2,
    boardDisplayWidth + 2));

    // 4. 显示游戏状态信息（错误提示、胜负等）
    showGameStatus(screen.offset(boardDisplayHeight + 2, 0));

    // 5. 显示当前游戏支持的命令列表
    gameViewer.showCommandList(screen.offset(boardDisplayHeight + 4, 0),
    playground);

    screen.outputToConsole(); // 将虚拟屏幕内容输出到真实控制台
}
// ... run() 方法包含主循环，调用 clearScreen() 和 displayPlayground() ...
}

```

设计思路: **PlayGroundViewer** 负责在控制台展示整个游戏界面。**displayPlayground** 方法是核心渲染函数，它将界面划分为几个区域（棋盘、玩家信息、游戏列表、状态信息、命令提示），并调用各自的辅助方法或 **GameViewer** 的方法将内容绘制到一个虚拟的 **RawScreen** 对象上。最后，**RawScreen** 的内容被一次性输出到控制台，模拟了界面的刷新。这种方式有助于组织复杂的文本UI布局。

Lab6与Pj复用情况分析

核心领域逻辑的高度复用：最为显著的复用体现在项目的核心——**domain**包。这部分代码封装了所有棋类游戏的通用规则、棋盘结构、棋子定义以及具体游戏的逻辑实现。

1. **游戏核心接口与类：**如 **Game** 接口 (`lab/domain/game/Game.java`) 定义了所有游戏必须遵循的统一契约，**Board** 类 (`lab/domain/board/Board.java`) 及其相关的 **Location**、**PieceColor** 等基础构建模块，这些源自Lab6的设计被完整地继承到了GUI版本中。这充分证明了项目在Lab6阶段就构建了与表现层无关的、高度内聚的领域模型。
2. **具体游戏逻辑的直接共享：**诸如 **PeaceGame** (`lab/domain/game/PeaceGame.java`)、**ReversiGame** (`lab/domain/game/ReversiGame.java`) 和 **GomokuGame** (`lab/domain/game/GomokuGame.java`) 等具体游戏的实现代码，几乎无需修改或只需少量适配，即可在GUI版本中直接使用。这得益于我在设计这些游戏类时，严格遵循了面向接口编程和逻辑封装的原则。
3. **游戏工厂的运用：****GameFactory** (`lab/domain/game/GameFactory.java`) 的设计 (在GUI版本的 **GameStateManager** (`lab/gui/models/GameStateManager.java` line 122) 和 **GameListView** (`lab/gui/views/GameListView.java` line 75) 中均有体现) 在Lab6基础上进一步完善，它使得游戏对象的创建更加灵活和解耦，无论是新游戏创建还是从状态恢复，都体现了良好的设计。

面向GUI的适配与演进：虽然核心逻辑得以重用，但为了适应GUI的特性，部分Lab6中的概念和组件也得到了巧妙的适配和功能增强：

1. **游戏会话管理：**GUI版本中的 **GameSession** 类 (`lab/gui/models/GameSession.java`) 可以看作是Lab6中 **PlayGround** 类 (`lab6/console/playground/PlayGround.java`) 的演进版本。它继承了管理多个游戏实例和玩家信息的职责，并针对JavaFX框架的特性 (如 **ObservableList**、**IntegerProperty**) 进行了优化，以支持GUI的数据绑定和动态更新。
2. **游戏状态持久化：****GameStateManager** (`lab/gui/models/GameStateManager.java`) 和 **SerializableGame** (`lab/gui/models/SerializableGame.java`) 的引入，为GUI版本提供了更完善和健壮的游戏状态保存与加载机制。这套机制能够处理更复杂的游戏属性 (如 **additionalAttributes**)，是在Lab6可能存在的简单持久化方案基础上的重要升级。
3. **命令处理的演化：**Lab6中基于文本的 **Command** 接口 (`lab6/console/command/Command.java`) 及其实现，在GUI版本中转变为通过事件处理器 (如 **MainController** (`lab/gui/controllers/MainController.java`) 中的 **handleBoardClick**) 与用户交互。尽管交互方式改变，但这些处理器最终调用的仍是复用自Lab6的领域逻辑。值得一提的是，GUI版本中的回放功能 (**processPlaybackCommand** in `MainController.java` line 387) 依然能够解析和处理命令字符串，这间接体现了Lab6命令设计的合理性和对核心操作的良好抽象。

Lab6代码可大量复用原因分析

1. **关注点分离 (Separation of Concerns):** 我成功地将游戏的核心业务逻辑 (Domain) 与用户界面 (Presentation) 彻底分离。Lab6的领域模型不依赖于任何控制台特定的API, 这为后续平滑迁移到GUI奠定了坚实基础。
2. **面向接口编程:** `Game` 接口 (`lab/domain/game/Game.java`) 的使用是典范。GUI层通过此接口与不同类型的游戏进行交互, 无需关心其具体实现, 大大增强了系统的灵活性和可扩展性。
3. **模块化设计:** 各个游戏逻辑模块化, 使得它们可以作为独立的单元被复用和集成。

添加新游戏类型相关分析

1. 核心领域模型的扩展 (Domain Layer)

- **实现标准游戏接口 (`Game`):**
 - 项目的核心优势在于定义了统一的 `Game` 接口。要添加2048游戏, 首先需要创建一个新的 `Game2048` 类并实现此接口。
 - `Game` 接口规范了游戏的基本行为, 如 `getGameType()`, `isOver()`, `getCurrentPlayer()`, `getBoard()`。对于2048, 这些方法需要根据其特有规则实现。例如, `getBoard()` 会返回一个表示数字网格的 `Board` 实例, 可能需要 `Board` 类支持存储非传统棋子信息 (如数字), 或者 `Game2048` 内部进行转换。
 - **可扩展的动作处理:** 2048游戏的核心操作是上、下、左、右滑动, 而非传统的“落子”。`Game` 接口中的 `getSupportedActions()` 和 `executeAction(String action, Object... params)` 方法为此类非标准交互提供了完美的扩展点。`Game2048` 将声明支持 "moveUp", "moveDown" 等操作, 并在 `executeAction` 中实现这些滑动逻辑。这避免了修改 `Game` 接口本身, 体现了对扩展开放、对修改关闭的原则。
 - 游戏得分等特定信息可以通过 `Game` 接口提供的 `getPlayerScore()` 等默认方法或通过 `getAdditionalAttributes()` 机制进行管理。
- **利用游戏工厂 (`GameFactory`):**
 - `GameFactory` 的设计是项目可扩展性的关键。通过其注册机制 (`register` 方法), 添加新的2048游戏类型只需一行代码, 将 "2048" 标识符、显示名称及 `Game2048` 的构造函数引用注册到工厂中。
 - 当系统需要创建新游戏时, 会通过工厂按类型标识符动态创建相应的游戏实例, 无需硬编码或大量的 `if-else` 判断, 大大简化了新游戏类型的集成。

2. 用户界面层的适配 (Console & GUI Layers)

- **控制台界面 (Console Layer):**
 - **视图适配 (`GameViewer`):** 需要为2048创建一个新的视图类, 例如 `Game2048View`, 继承自抽象的 `GameViewer`。它将负责在控制台以数字矩阵的形式展示2048的棋盘状态, 这可能需要自定义 `BoardViewer` 的字符渲染逻辑或在 `Game2048View` 中直接处理。

- **命令扩展 (Command):** 2048的滑动操作需要在控制台通过命令触发。可以创建新的 `Command` 实现 (如 `MoveCommand`)，这些命令会调用 `Game2048` 实例的 `executeAction` 方法。`PlayGround` 会根据当前游戏类型提供相应的命令列表。
- **图形用户界面 (GUI Layer):**
 - **控制器逻辑 (MainController):**
 - 在 `main.fxml` 中添加一个“开始2048游戏”的按钮，并在 `MainController` 中为其添加事件处理，该处理会调用 `GameFactory` 创建2048游戏实例。
 - 2048游戏通常通过键盘方向键操作。`MainController` 需要添加键盘事件监听逻辑。当当前游戏是2048时，捕获方向键输入并调用当前游戏对象的 `executeAction` 方法。
 - **棋盘视图 (BoardView):**
 - 现有的 `BoardView` 主要设计用于显示棋子。为了显示2048的数字瓦片，可能需要扩展 `drawPiece` 方法，使其能够根据 `PieceColor` 的特殊值（或从游戏对象获取的额外信息）渲染数字和不同的背景色。如果差异过大，也可以考虑为2048创建一个专门的视图组件，但理想情况下，通用组件应具备一定的灵活性。
 - 点击交互 (`handleBoardClick`) 对于2048可能不适用，主要交互将通过键盘。
 - **游戏信息展示:** 2048的得分等信息可以通过 `MainController` 中的 `updateSpecialGameInfo` 方法，利用 `Game` 接口提供的 `getPlayerScore` 或 `getAdditionalAttributes` 来获取并展示。

3. 游戏状态的持久化 (GUI Models)

- **利用可扩展的序列化模型 (SerializableGame):**
 - 项目的状态管理通过 `GameStateManager` 和 `SerializableGame` 实现，这部分设计也考虑了扩展性。
 - `SerializableGame` 类包含一个 `additionalAttributes` (类型为 `Map<String, Serializable>`) 字段。这是存储特定游戏状态（如2048的数字网格布局、当前分数）的关键。
 - `Game2048` 类需要实现 `Game` 接口中的 `getAdditionalAttributes()` 方法，将自己特有状态打包到Map中返回。同时，也需要实现 `restoreAdditionalAttributes(Map<String, Serializable> attributes)` 方法，以便在加载游戏时从Map中恢复状态。
 - 这种设计使得核心的保存和加载逻辑（在 `GameStateManager` 中）无需为每种新游戏做大的修改，只需依赖 `Game` 接口定义的方法即可。

项目运行截图

1. 初始界面

A B C D E F G H

1

2

3

4

5

6

7

8

游戏1

黑棋 [Jia] ●

白棋 [Ran]

演示模式

退出游戏

游戏列表

1. 和平棋

2. 黑白棋

3. 五子棋

添加新游戏

和平棋

黑白棋

五子棋

A B C D E F G H

1

2

3

4

5

6

7

8

游戏2

黑棋 [Jia] ● 2

白棋 [Ran] 2

跳过本轮

演示模式

退出游戏

游戏列表

1. 和平棋

2. 黑白棋

3. 五子棋

添加新游戏

和平棋

黑白棋

五子棋

A B C D E F G H I J K L M N O

1

2

3

4

5

6

7

8

9

A

B

C

D

E

F

游戏3

黑棋 [Jia] ● *2

白棋 [Ran] ● *3

当前回合: 1

使用炸弹

演示模式

退出游戏

游戏列表

1. 和平棋

2. 黑白棋

3. 五子棋

添加新游戏

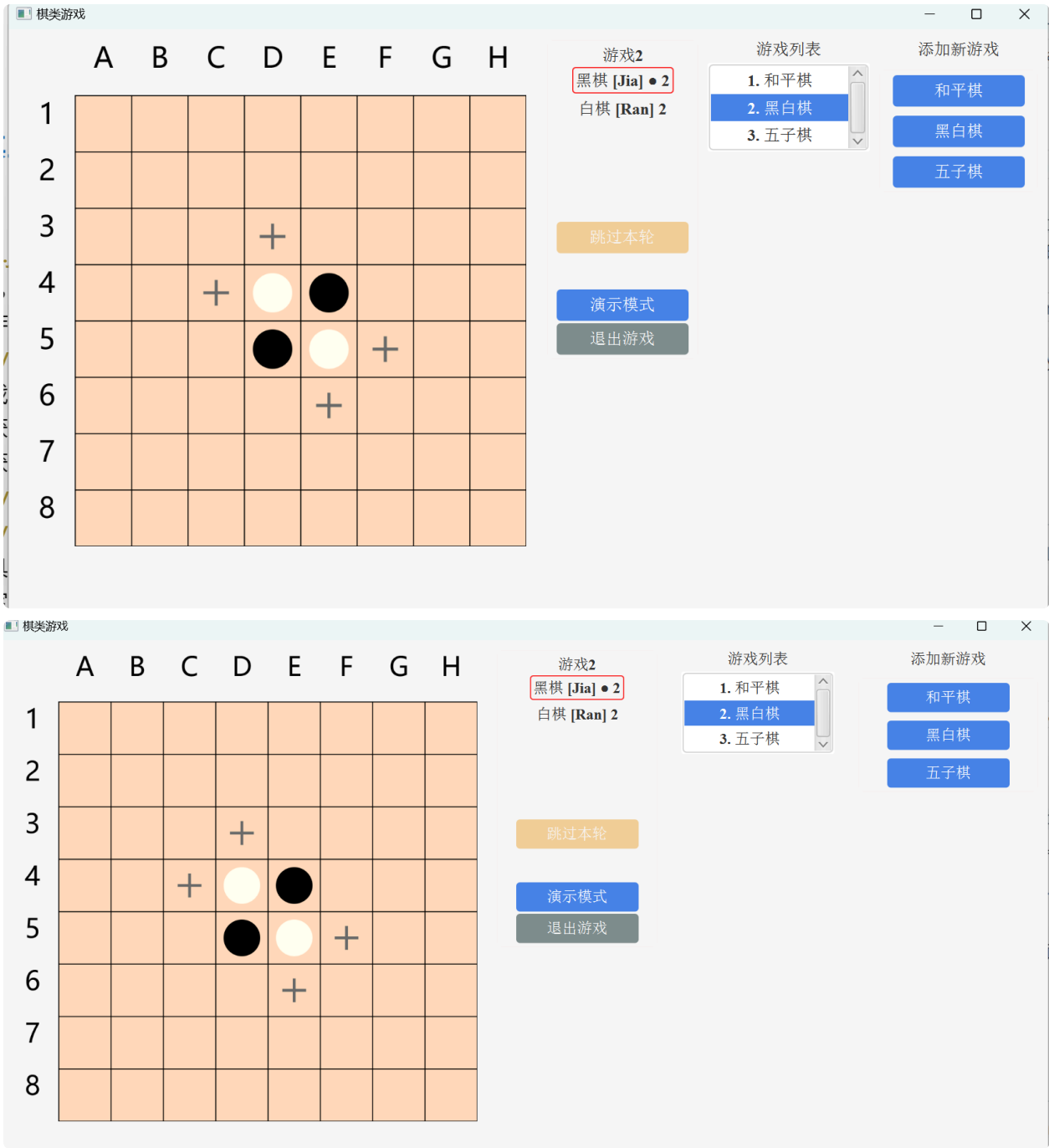
和平棋

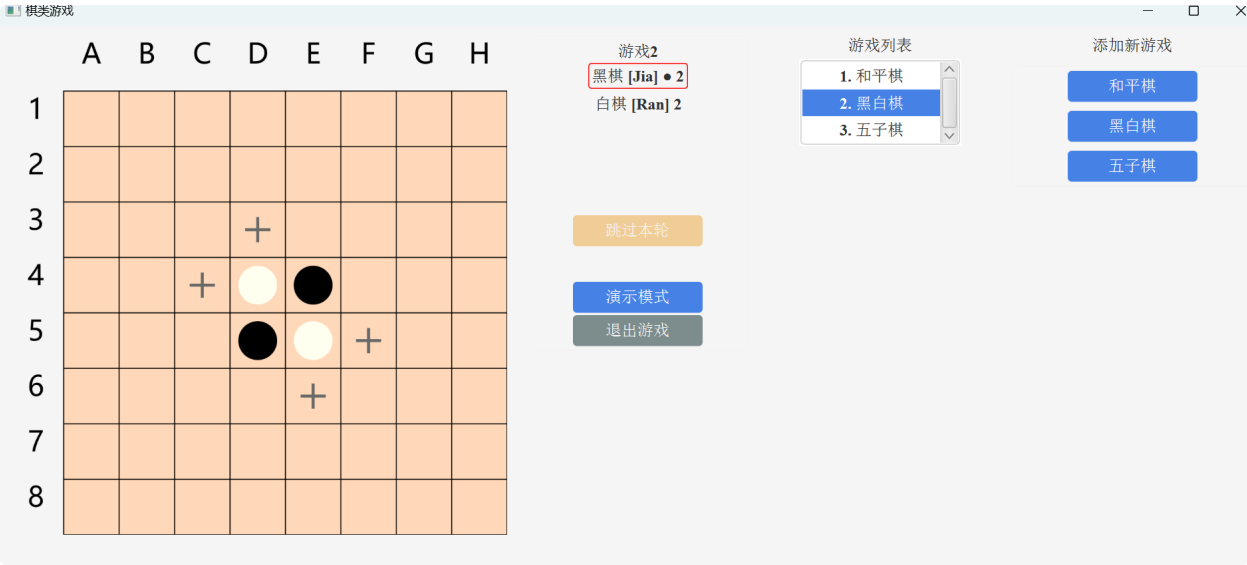
黑白棋

五子棋

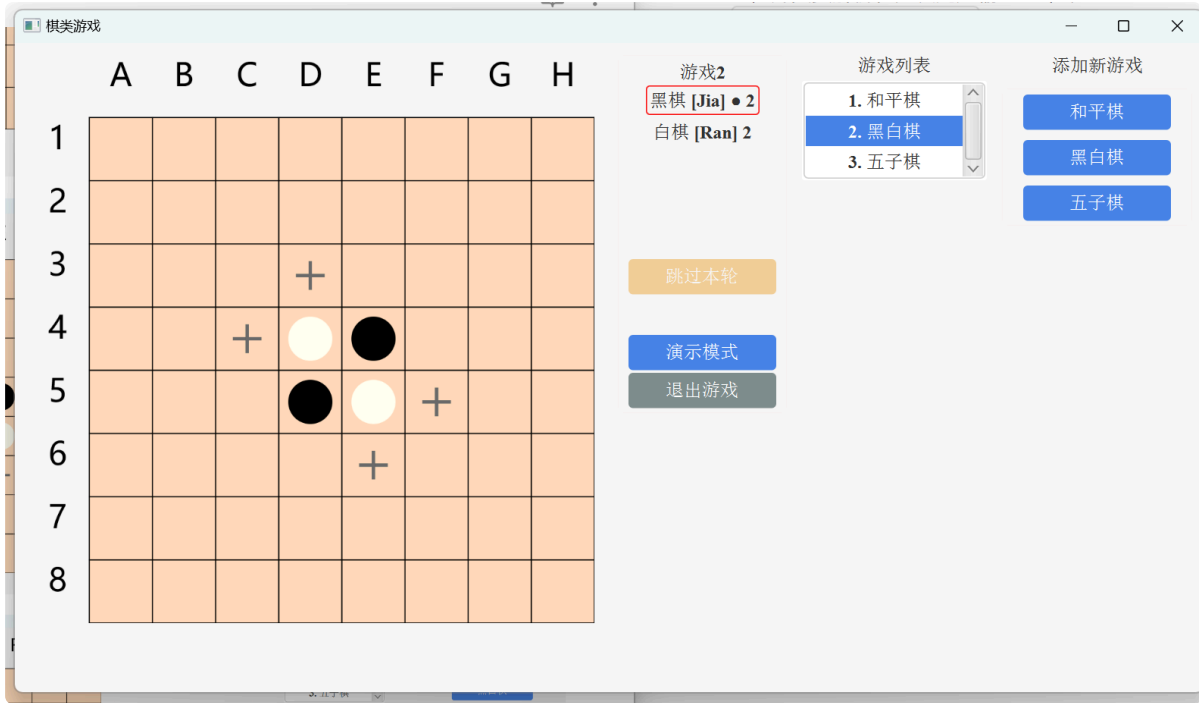
2. 调整窗口界面大小时的效果

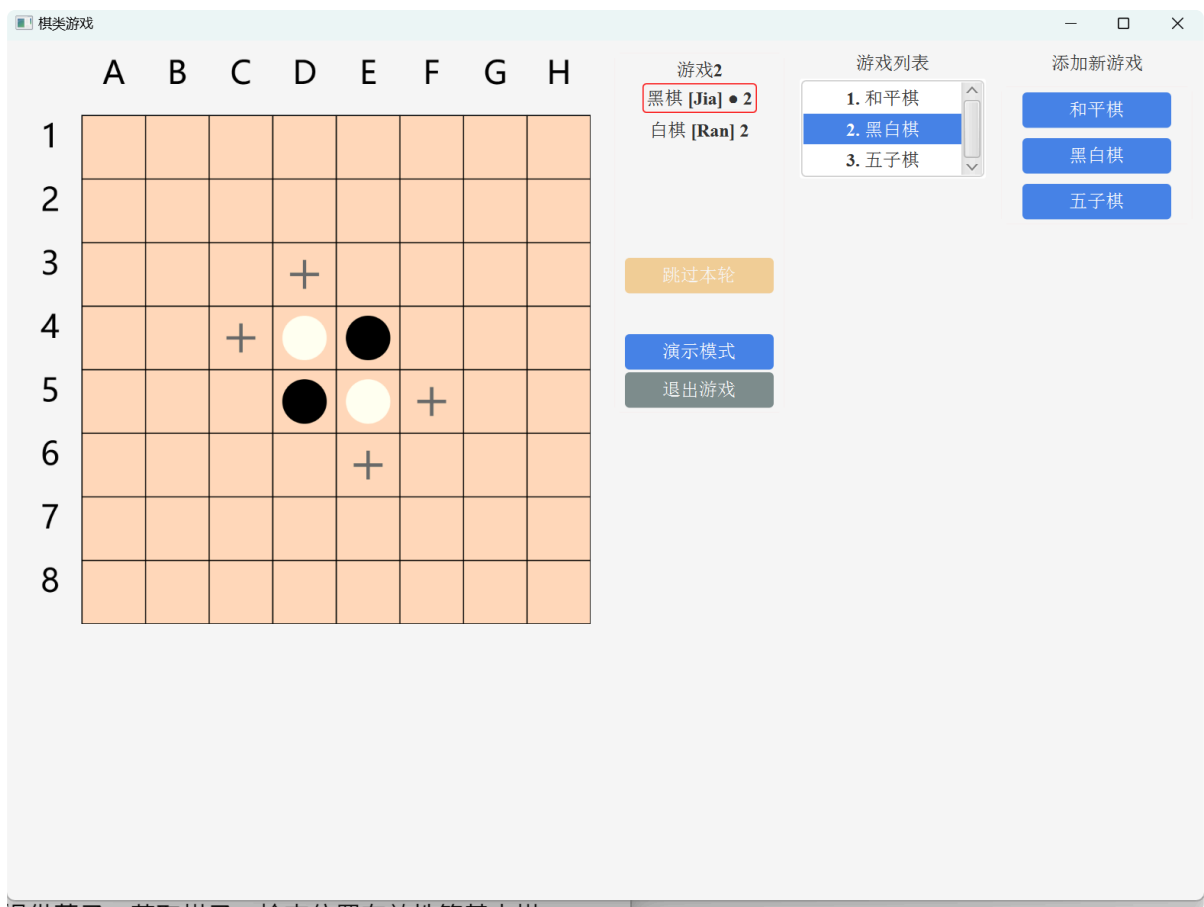
- 宽度调整





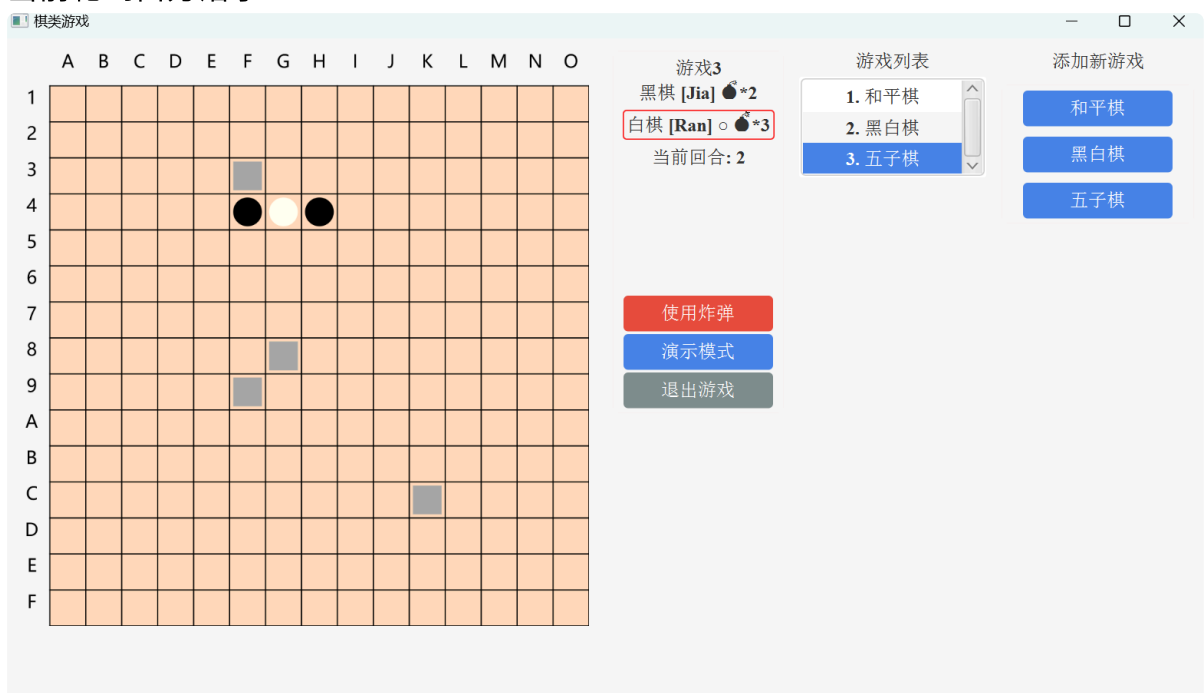
高度调整



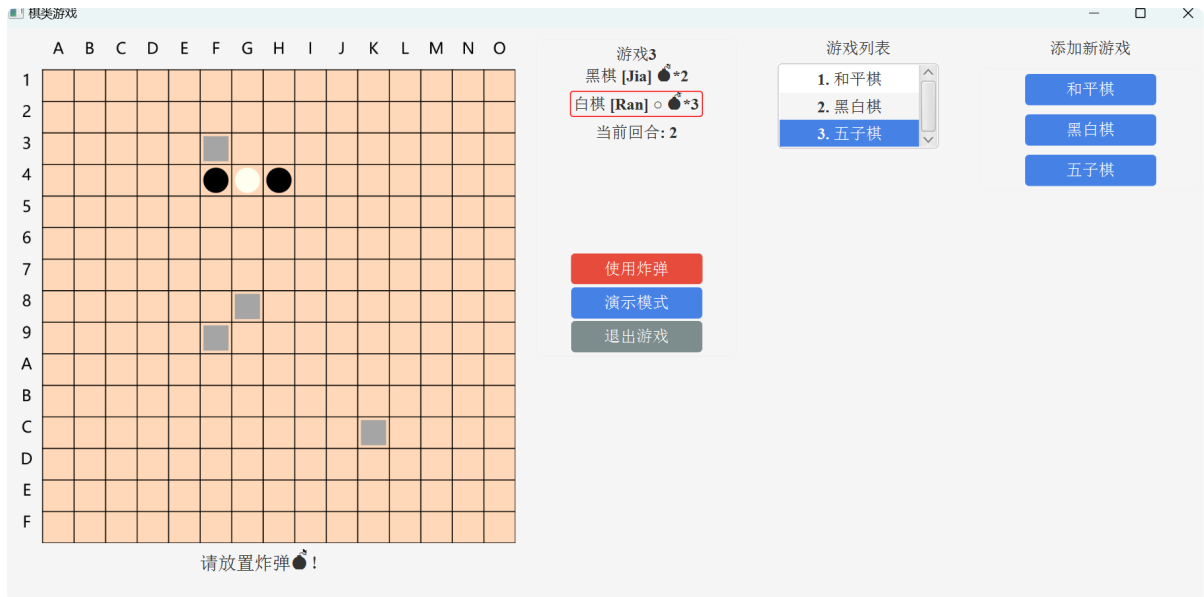


3. Gomoku放置炸弹

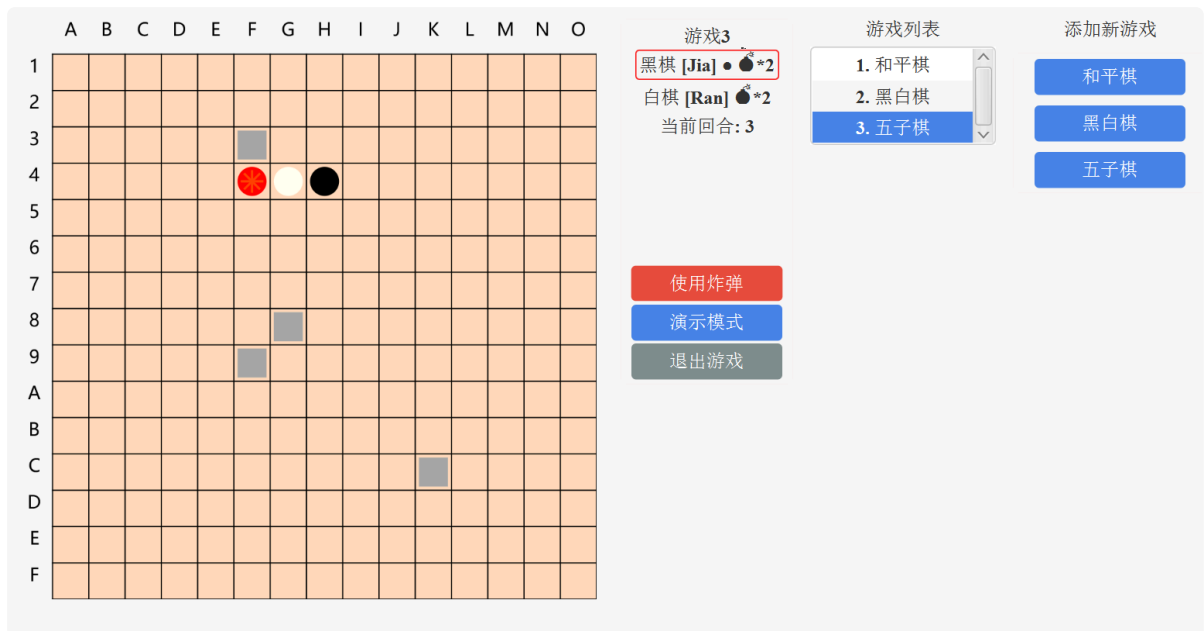
- 当前轮到白方落子



- 白方点击“使用炸弹”按钮

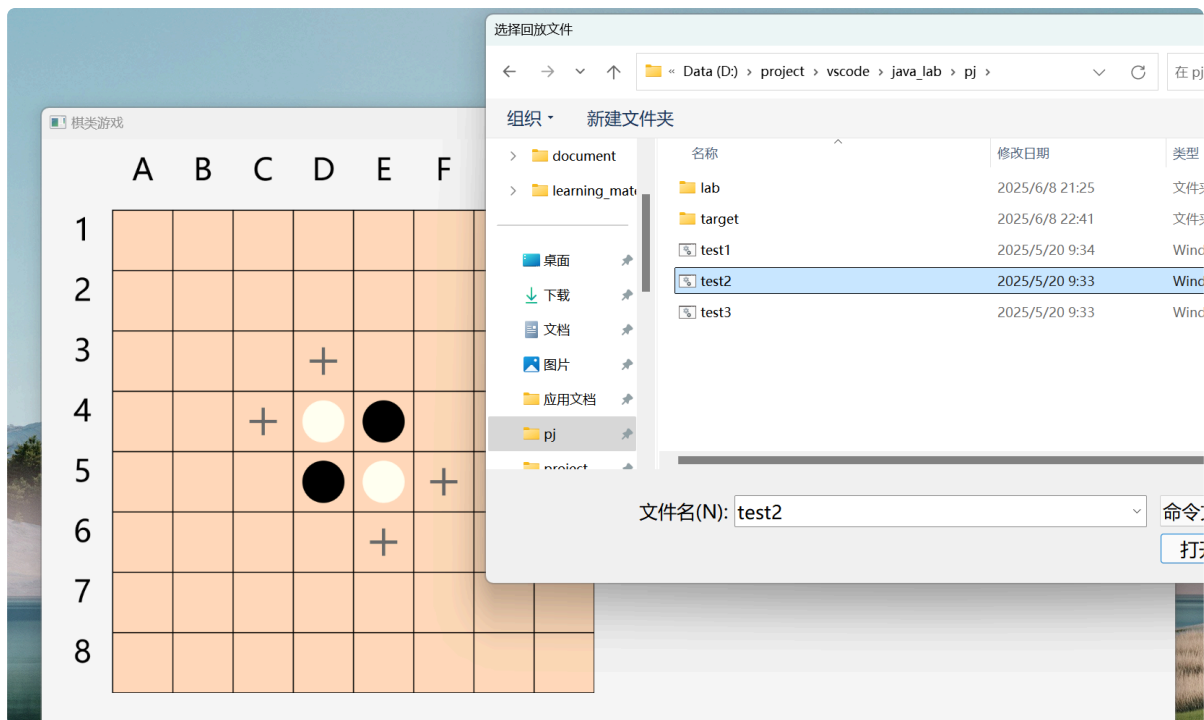


- 点击4F放置炸弹

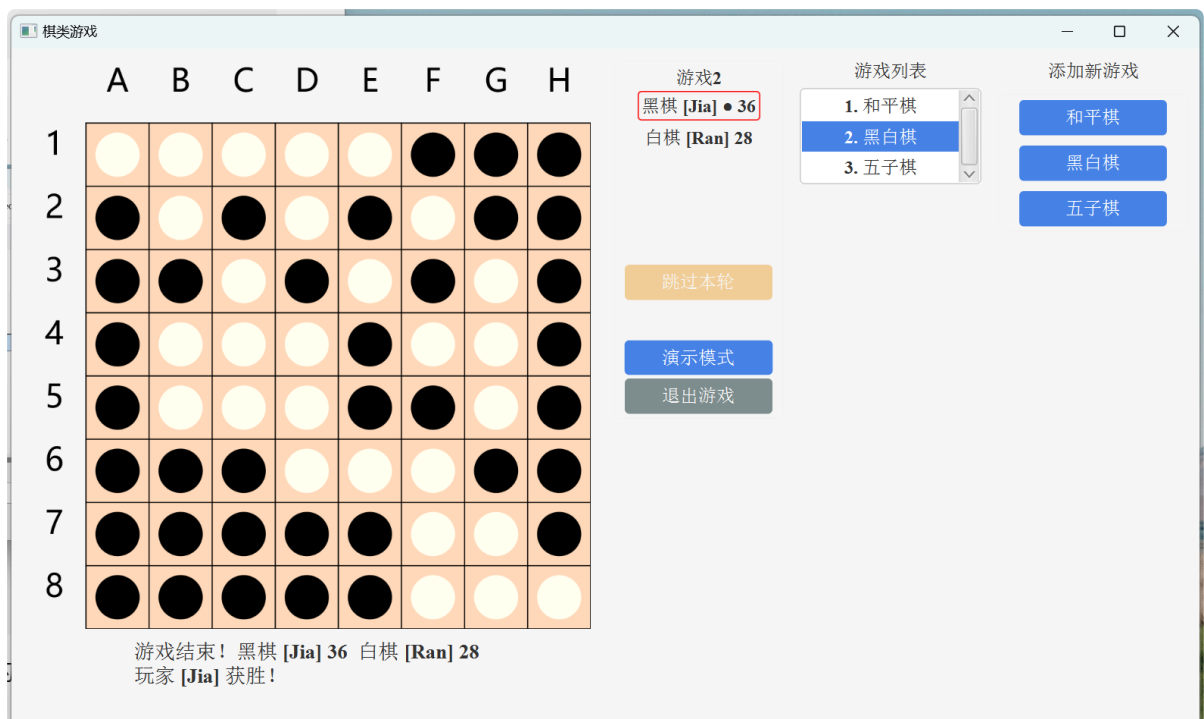


3. 演示模式（以Reversi为例）

- 点击“演示模式”按钮，选择对应的指令文件



- 每间隔一段时间执行一步指令并显示对应的GUI状态。最终结果如下：



- 演示模式执行之后，应当可以继续下棋或者执行其他操作（例如新建游戏）

